# Shortest Path Algorithms

Shortest Path Algorithms are used to find a path between two vertices (or a single vertex and all other vertices) in a graph such that the sum of the weights of its edges is minimized.

| Algorithm | Primary Use | Graph Type | Negative Weights | Time Complexity |
|---|---|---|---|---|
| **Dijkstra's** | Single-Source Shortest Path | Weighted, Directed/ Undirected | **No** (Must be non-negative) | $O(E + V \log V)$ (with Min Heap) |
| **Bellman-Ford** | Single-Source Shortest Path | Weighted, Directed/ Undirected | **Yes** (Detects negative cycles) | $O(V \cdot E)$ |

---

## Dijkstra's Algorithm

**Dijkstra's Algorithm** finds the shortest path from a single source node to all other nodes in a graph with **non-negative** edge weights.

- **Principle:** It uses a **greedy approach**, always selecting the unvisited node that currently has the smallest known distance from the source.
- **Data Structure:** A **Min Priority Queue (Min Heap)** is crucial for efficiently retrieving the node with the minimum distance.
- **Limitation:** It **fails** if the graph contains **negative cycles** or **negative edge weights**.

### C++ Code (Conceptual Core)

```
#include <queue>
#include <vector>
#include <utility> // For std::pair

// pair<distance, vertex>
void dijkstra(int V, const std::vector<std::pair<int, int>> adj[], int
    std::priority_queue<std::pair<int, int>,
                        std::vector<std::pair<int, int>>,
                        std::greater<std::pair<int, int>>> pq;

    std::vector<int> dist(V, INT_MAX);

    dist[start] = 0;
    pq.push({0, start}); // {distance, vertex}

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        // Check if we found a shorter path already (stale entry)
        if (d > dist[u]) continue;
```

```
        // Relaxation step applied to all neighbors v of u
        for (auto& edge : adj[u]) {
            int v = edge.first;
            int weight = edge.second;

            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }
}
```

---

# Bellman-Ford Algorithm

**Bellman-Ford Algorithm** finds the shortest path from a single source node to all other nodes and can handle **negative edge weights**.

- **Principle:** It iteratively relaxes all edges in the graph $V-1$ times.
- **Negative Cycle Detection:** A final $V$-th iteration is performed. If any distance is still reduced (relaxed), a **negative cycle** exists.
- **Time Complexity:** $O(V \cdot E)$.

## C++ Code (Conceptual Core)

```
#include <vector>
#include <tuple> // For std::tuple (u, v, weight)

// tuple<u, v, weight>
void bellmanFord(int V, const std::vector<std::tuple<int, int, int>>& e
    std::vector<int> dist(V, INT_MAX);
    dist[start] = 0;

    // 1. Relax all edges V - 1 times
    for (int i = 0; i < V - 1; ++i) {
        for (const auto& edge : edges) {
            int u = std::get<0>(edge);
            int v = std::get<1>(edge);
            int weight = std::get<2>(edge);

            // Relaxation step
            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
            }
        }
    }

    // 2. Check for negative cycles (V-th iteration)
    bool negativeCycle = false;
    for (const auto& edge : edges) {
```

```
            int u = std::get<0>(edge);
            int v = std::get<1>(edge);
            int weight = std::get<2>(edge);

            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                // Negative Cycle detected
                negativeCycle = true;
                break;
            }
        }

        if (negativeCycle) {
            std::cerr << "Graph contains a negative weight cycle!" << std::
            // Handle error...
        }
    }
```

---

# ✧ The Relaxation Step

The **Relaxation Step** is the core mechanism used in many shortest path algorithms (Dijkstra's, Bellman-Ford, etc.) to iteratively update the shortest distance to a vertex.

- **Definition:** Given an edge $u \to v$ with weight $w(u, v)$, relaxation checks if the path to $v$ can be shortened by going through $u$.
- **Variables:**
  - $\text{dist}[u]$: The currently known shortest distance from the source to $u$.
  - $\text{dist}[v]$: The currently known shortest distance from the source to $v$.
- **Condition & Update:** If the current distance to $u$ plus the weight of the edge to $v$ is less than the current distance to $v$, update $\text{dist}[v]$. $$\text{if } \text{dist}[u] + w(u, v) < \text{dist}[v]$$ $$\text{then } \text{dist}[v] = \text{dist}[u] + w(u, v)$$
- **Effect:** It ensures that the distance estimate $\text{dist}[v]$ is constantly refined and becomes the true shortest distance only when no further improvements can be made.